

COLLEGE PROJECT — COMPUTER SCIENCE



AI-Based Plagiarism, Paraphrase & Authenticity Checker

A Java + Spring Boot academic integrity system

React · Java · Spring Boot · MySQL · Apache PDFBox / POI

Submitted for
B.Tech / BCA Final Year Project

Technology
React · Spring Boot · MySQL

Domain
Academic Integrity / NLP / AI

01 Abstract

This project presents a Java-based AI-integrated academic integrity system designed to detect copied, paraphrased, and suspicious research content. The application accepts text or uploaded documents (PDF / DOCX), compares them against a reference database, calculates similarity scores, highlights matching sections, and provides AI-generated rewriting suggestions. A research paper authenticity check analyses section-wise structure and citation patterns to flag duplicated or suspicious submissions — making this system significantly more advanced than a conventional string-matching plagiarism checker.

02 Introduction

Plagiarism is a persistent challenge in academic writing. Students and researchers sometimes submit content that is directly copied, heavily paraphrased, or reused from published work without proper acknowledgment. While basic string-matching tools detect verbatim copies, they consistently fail against disguised plagiarism — content that has been reworded, translated, or restructured to conceal its original source.

This project addresses that gap by combining lexical matching, semantic similarity analysis, and citation-aware detection in a single Java application. The result is a practical, AI-enabled integrity tool suitable for deployment in educational institutions.

03 Problem Statement

Existing plagiarism tools suffer from three core limitations:

- **String matching only:** Tools like basic Jaccard or n-gram checkers find exact copies but miss paraphrased or translated content.
- **No section-level awareness:** Abstract and conclusion duplication across research papers goes undetected without section-specific analysis.
- **Citation blindness:** Papers that omit or manipulate citations to mask idea reuse are not flagged by traditional tools.

A smarter system combining lexical matching, semantic NLP, and citation-pattern analysis is required to reliably detect both direct and disguised plagiarism.

04 Objectives

- Detect directly copied text from uploaded documents.
- Identify paraphrased content using meaning-based (semantic) similarity comparison.
- Flag suspicious or duplicated research papers through section-wise analysis.
- Display overall similarity percentage and highlight matched sentence segments.
- Generate AI-powered rewriting suggestions for flagged content.
- Produce a downloadable authenticity / risk report for academic review.

05 Scope of the Project

The system is targeted at students, faculty members, and academic institutions for reviewing submissions before final acceptance. It covers the following use cases:

- Student assignment and thesis submission review.
- Research paper pre-submission check against a curated reference corpus.
- Batch processing of departmental submissions with a dashboard report.

Future scope includes integration with institutional LMS platforms, support for multilingual plagiarism detection, AI-generated text detection, and cloud-based document storage with cross-institution sharing.

06

Core Features

Feature	Module	Description
File Upload	React	Accepts pasted text, PDF, and DOCX files via a clean upload interface.
Plagiarism Detection	Similarity Engine	Compares submitted text against stored reference documents using TF-IDF and cosine similarity.
Paraphrase Detection	NLP / Semantic AI	Identifies reworded sentences that preserve original meaning using sentence embeddings.
Authenticity Check	Citation Module	Analyses title, abstract, introduction, conclusion, and citation patterns for suspicious reuse.
Rewriting Suggestions	Suggestion Engine	Provides AI-generated alternate phrasing for flagged sentences to help improve originality.
Final Report	Reporting Module	Generates a downloadable PDF / HTML report with similarity score, source matches, and warnings.

07

System Workflow

1	Upload	User uploads a PDF / DOCX file or pastes text directly.
2	Extract	Apache PDFBox / Apache POI extracts raw text from the file.
3	Clean & Split	Text is normalised and divided into sentence-level chunks.
4	TF-IDF Compare	Each chunk is compared with the reference corpus using TF-IDF vectorisation.
5	Semantic Check	High-similarity chunks undergo a deeper semantic similarity pass via sentence embeddings.
6	Citation Analysis	Citation structure and section layout are evaluated for suspicious patterns.
7	Score & Flag	A suspicion score is assigned and matching sentences are highlighted.
8	Report	A final authenticity / risk report is generated and made available for download.

08 Module Design

User Module

Login · Registration · Upload · Report access

Document Processing Module

PDF/DOCX extraction via Apache PDFBox and Apache POI

Similarity Engine Module

TF-IDF · Cosine similarity · Sentence embeddings

Citation Analysis Module

Citation pattern checks · Section-wise alerts

Suggestion Engine Module

AI-powered rewrite suggestions for flagged sentences

Reporting Module

Downloadable PDF/HTML reports with scores and highlights

09 Technology Stack

Layer	Technology	Purpose
Backend	Java 17, Spring Boot 3	Core business logic and REST APIs
Frontend	React, Bootstrap / Tailwind CSS	Document upload and report display UI
Database	MySQL, Spring Data JPA, Hibernate	Persistent storage for documents and results
Document Parsing	Apache PDFBox, Apache POI	Extract text from PDF and DOCX files
Similarity	TF-IDF, Cosine Similarity, Sentence Embeddings	Lexical and semantic comparison
AI Integration	Spring AI or REST-based AI service	Paraphrase detection and rewrite suggestions
Security	Spring Security	Authentication and authorisation

10

Database Design

The system uses a MySQL relational database with the following tables:

Table	Key Columns	Purpose
users	id, name, email, password_hash, role, created_at	Stores user credentials and role (student / faculty / admin).
documents	id, user_id, title, file_path, upload_date, status	Metadata for each uploaded document linked to a user.
document_sections	id, document_id, section_type, content	Extracted sections: abstract, introduction, conclusion, etc.
similarity_results	id, document_id, source_doc_id, score, flagged_sentences	Similarity scores and matched sentence references.
citation_warnings	id, document_id, warning_type, description	Citation-related alerts and suspicious academic patterns.
reports	id, document_id, authenticity_score, report_path, created_at	Final report records and downloadable file paths.

11

Expected Output

- **Overall similarity percentage** — a single aggregated score shown prominently.
- **Highlighted copied sentences** — red highlight for direct matches.
- **Highlighted paraphrased sentences** — yellow highlight for semantic matches.
- **Matched source documents** — list of reference documents with per-source scores.
- **Citation warnings** — list of uncited or suspiciously cited sentences.
- **Authenticity / risk score** — a combined score (e.g. High Risk / Medium / Low Risk).
- **Downloadable report** — PDF or HTML export for lecturer review.

12

Hardware & Software Requirements

Hardware

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD equivalent	Intel Core i5 or better
RAM	4 GB	8 GB or more
Storage	20 GB free	50 GB SSD
Network	Required for AI REST calls	Broadband

Software

- Java Development Kit (JDK) 17 or higher
- Apache Maven or Gradle (build tool)

- Spring Boot 3.x with Spring Web, Spring Security, Spring Data JPA
- MySQL 8.x database server
- Apache PDFBox 3.x and Apache POI 5.x
- Node.js and npm (for the React frontend build)
- Spring AI or a configured REST-based AI API endpoint
- IntelliJ IDEA / Eclipse (development IDE)

13

Algorithm Overview

The core plagiarism detection pipeline operates as follows:

Input	Accept document via upload or text paste; extract raw text using PDFBox / POI.
Preprocessing	Lowercase, remove stop-words, tokenise into sentences and n-gram chunks.
TF-IDF Vectorisation	Represent each chunk as a TF-IDF vector against the reference corpus vocabulary.
Cosine Similarity	Compute cosine similarity between each chunk vector and all reference document vectors.
Threshold Check	Chunks above a configurable threshold (e.g. 0.7) are flagged as potential matches.
Semantic Pass	Flagged chunks are re-evaluated using sentence embeddings for paraphrase detection.
Citation Check	Citation count, format, and placement are analysed against expected academic patterns.
Score Aggregation	A weighted suspicion score is computed: 60% lexical + 30% semantic + 10% citation.
Output	Highlighted document view, matched source list, citation warnings, and downloadable report.

14

Future Scope

- Cloud-based document storage and cross-institution reference corpus sharing.
- AI-generated text detection (GPT / LLM output identification).
- Multilingual plagiarism detection supporting regional academic languages.
- Author identity verification using writing-style fingerprinting.
- Integration with LMS platforms (Moodle, Google Classroom).
- Institutional dashboard with batch submission analytics and trend reports.
- Mobile app for on-the-go document submission and result viewing.

15

Conclusion

This project delivers a practical, modern solution for academic content verification. By combining Java, Spring Boot, Apache document parsing libraries, TF-IDF cosine similarity, and AI-powered semantic analysis, the system addresses the core shortcomings of conventional plagiarism checkers. The addition of section-wise paper analysis and citation-pattern evaluation makes it uniquely suited for research paper authenticity checking — going beyond simple string matching to deliver genuinely intelligent academic integrity support.

"This project is a Java-based AI-integrated academic integrity system that detects copied and paraphrased content, compares research papers with a reference database, analyses citation patterns, and generates an authenticity report."

— Suggested viva description